SDS - REQ-IF-003 공지 · 일정 REST API 제공

작성일 2026-10-17 · 근거: 요구사항정의서_엑셀양식_v3_1_3.xlsx (F열) · SRS 없음 · 대상: D:₩legacy-portal

1. 개요

항목	내용
요구사항 ID	REQ-IF-003
요구사항명	공지 · 일정 REST API 제공
대 / 중 / 소분류	PORTAL / Notice / API
우선순위	Medium
진행현황	기획중
수용여부 · 요청자	수용 · 포털팀 (미정)
엑셀 비고	미구현(컨트롤러 없음). 신규 요구.
요약	공지(작성 · 게시 · 목록)와 일정(등록 · 확정 · 내 일정)을 REST API 로 연다. 현재 소스에는 NoticeService · ScheduleService 만 있고 컨트롤러 · DTO 가 없어 API 는 전부 미구현이다. 본 설계서는 결재 API(R11)와 같은 방식(record DTO · 서비스 위임 · 엔티티 비노출)으로 URL · DTO · 응답을 확정하고, 기존 서비스의 업무 규칙 · 감사로그 · 메일 · 조용한 무시(null 반환) 동작은 그대로 둔다.

2. 근거

SRS 없음 (엑셀 F열 · 소스만으로 분석). docs₩SRS 폴더가 존재하지 않아 아래 F열 상세내용 원문과 D:₩legacy-portal 소스(src₩main · src₩test) · CLAUDE.md 불변 규칙만을 근거로 삼는다.

[개요]
서비스 계층에만 있는 공지 · 일정 기능을 화면과 외부에서 쓸 수 있도록 REST API 로 연다.

[상세 기능 요구사항]
1) 공지: 작성(POST /api/notices), 게시(POST /api/notices/{id}/publish), 목록 조회(GET /api/notices) 를 제공한다.
2) 일정: 등록(POST /api/schedules), 확정(POST /api/schedules/{id}/confirm), 내 일정 조회(GET /api/schedules/owner/{userId}) 를 제공한다.
3) 요청 · 응답은 결재 API 처럼 record DTO 로 받고 돌려준다.
4) 업무 규칙은 기존 NoticeService · ScheduleService 를 그대로 호출한다.

[입력 / 출력]
- 입력: 공지 · 일정 요청 JSON
- 출력: 공지 · 일정 응답 JSON

[제약 조건 / 비기능 요구사항]
- 엔티티를 응답으로 직접 노출하지 않는다.
- 기존 서비스의 업무 규칙 · 감사로그 · 메일 동작을 바꾸지 않는다.
- URL 은 설계 단계에서 확정한다. 위 경로는 초안이다.

[검증 기준]
- API 로 등록한 공지 · 일정이 서비스 특성화 테스트와 같은 결과를 낸다.
- 응답 JSON 에 엔티티 내부 필드가 아닌 DTO 필드만 나온다.

[작업 대상]

legacy-portal (D:₩legacy-portal) Spring Boot 3.2 / Java 21
src₩main₩java₩kr₩hull₩portal₩notice
src₩main₩java₩kr₩hull₩portal₩schedule

[Worker Settings]
- MainWorker : claude-main
- SubWorker : codex-critic
- MainWorker-SubAgent : backend-expert

3. 분석 결과

3.1 추적표 (F열 문장 ↔ 소스)

항목	요구사항(F열 문장)	SRS 근거	소스 위치	판정	비고
기능 1	공지: 작성(POST /api/notices), 게시(POST /api/notices/{id}/publish), 목록 조회(GET /api/notices) 를 제공한다.	SRS 없음	notice 패키지에 컨트롤러 없음 (Notice.java · NoticeRepository.java · NoticeService.java 3개뿐). 서비스: NoticeService.java:66 create, :78 publish. 목록 메서드 없음 (NoticeRepository.java:5 JpaRepository 상속만)	**미충족**	API 미구현. 목록 조회는 서비스에 메서드가 없어 읽기 메서드 추가 필요(4절)
기능 2	일정: 등록(POST /api/schedules), 확정(POST /api/schedules/{id}/confirm), 내 일정 조회(GET /api/schedules/owner/{userId}) 를 제공한다.	SRS 없음	schedule 패키지에 컨트롤러 없음. 서비스: ScheduleService.java:44 create, :78 confirm. 조회는 ScheduleRepository.java:7 findByOwnerId 만 있고 서비스 메서드 없음	**미충족**	API 미구현. 내 일정 조회용 서비스 읽기 메서드 추가 필요
기능 3	요청 · 응답은 결재 API 처럼 record DTO 로 받고 돌려준다.	SRS 없음	참조 패턴: approval/dto/CreateApprovalRequest.java:18-32 · ProcessRequest.java:15-20 · ApprovalResponse.java:16-44. notice · schedule 에는 dto 패키지 없음	**미충족**	결재 DTO 패턴(compact 생성자 기본값 · from() 변환)을 그대로 따른다
기능 4	업무 규칙은 기존 NoticeService · ScheduleService 를 그대로 호출한다.	SRS 없음	호출 주체(컨트롤러) 없음. 업무 규칙 위치: Notice.java:84-93 publish · Schedule.java:61-69 isCreatable · :95-100 overlaps · :108-117 confirm	**미충족**	컨트롤러는 서비스 public 메서드만 호출 - 규칙을 컨트롤러로 복제하지 않는다
입력	입력: 공지 · 일정 요청 JSON	SRS 없음	요청 DTO 없음	**미충족**	5절에서 필드 · 타입 확정
출력	출력: 공지 · 일정 응답 JSON	SRS 없음	응답 DTO 없음	**미충족**	5절에서 필드 · 타입 확정
제약 1	엔티티를 응답으로 직접 노출하지 않는다.	SRS 없음	공지 · 일정 API 자체가 없음. (참고: 범위 밖 UserController.java:23-25 는 User 엔티티를 직접 반환)	**확인 불가**	대상 API 부재로 판정할 대상이 없음. 설계에서 NoticeResponse · ScheduleResponse 로 강제

항목	요구사항(F열 문장)	SRS 근거	소스 위치	판정	비고
제약 2	기존 서비스의 업무 규칙 · 감사로그 · 메일 동작을 바꾸지 않는다.	SRS 없음	NoticeService.java:66-108 · ScheduleService.java:44-92 현행. 특성화 테스트 NoticeServiceCharacterizationTest 17개 · ScheduleServiceCharacterizationTest 24개 green (2026-10-17 ./mvnw test 144개 전부 통과)	**충족**	현행 서비스는 변경되지 않은 상태. API 추가 후에도 이 테스트 본문 그대로 green 이어야 함
제약 3	URL 은 설계 단계에서 확정한다. 위 경로는 초안이다.	SRS 없음	소스에 해당 URL 없음 (src₩main · src₩test 에서 /api/notices · /api/schedules Grep 결과 0건)	**확인 불가**	설계 단계 결정 사항 - 5절에서 초안 경로를 그대로 확정안으로 제시

3.2 판정 요약

충족 1 / 부분 충족 0 / 미충족 6 / 불일치 0 / 확인 불가 2 (총 9건)

3.3 검증 기준 ↔ 테스트

검증 기준(F열)	고정하는 테스트	비고
API 로 등록한 공지 · 일정이 서비스 특성화 테스트와 같은 결과를 낸다.	테스트 없음 (API 부재). 비교 기준이 될 서비스 테스트: NoticeServiceCharacterizationTest#공지를_생성하면_임시_상태0으로_입력값_그대로_저장된다, ScheduleServiceCharacterizationTest#일정을_생성하면_예정_상태0으로_입력값_그대로_저장된다	9절 NoticeControllerTest · ScheduleControllerTest 신설 제안
응답 JSON 에 엔티티 내부 필드가 아닌 DTO 필드만 나온다.	테스트 없음	응답 필드 개수 (jsonPath("$.*", hasSize(n)))로 고정 - ApprovalControllerTest#결재_생성_응답은_필드_12개의_레거시_JSON이다 와 같은 방식

4. 구성 요소 설계

구성 요소(신규/기존)	패키지	책임	비고
NoticeController (신규)	kr.hull.portal.notice	HTTP ↔ DTO 변환, NoticeService 호출만	@RestController @RequestMapping("/api/notices")
CreateNoticeRequest (신규 record)	kr.hull.portal.notice.dto	공지 작성 요청 본문, pinned 생략 시 false	compact 생성자에서 기본값
PublishNoticeRequest (신규 record)	kr.hull.portal.notice.dto	게시 요청 본문(userId)	
NoticeResponse (신규 record)	kr.hull.portal.notice.dto	공지 응답, from(Notice) 한 곳에서만 변환	status · category 는 정수 code
ScheduleController (신규)	kr.hull.portal.schedule	HTTP ↔ DTO 변환, ScheduleService 호출만	@RequestMapping("/api/schedules")

구성 요소(신규/기존)	패키지	책임	비고
CreateScheduleRequest (신규 record)	kr.hull.portal.schedule.dto	일정 등록 요청 본문	
ConfirmScheduleRequest (신규 record)	kr.hull.portal.schedule.dto	확정 요청 본문(userId)	
ScheduleResponse (신규 record)	kr.hull.portal.schedule.dto	일정 응답, from(Schedule)	status 정수 code
NoticeService (기존)	kr.hull.portal.notice	업무 흐름(변경 없음) + 읽기 메서드 1개 추가	기존 public 시그니처 유지
ScheduleService (기존)	kr.hull.portal.schedule	업무 흐름(변경 없음) + 읽기 메서드 1개 추가	기존 public 시그니처 유지

주요 시그니처

```
// 기존 - 시그니처 현행 유지 (변경 금지)
public Notice   NoticeService.create(String title, String content, int category, Long writerId, boolean pinned)
public void     NoticeService.publish(Long id, Long userId)
public String   NoticeService.statusLabel(Notice notice)
public Schedule ScheduleService.create(String title, Long ownerId, LocalDateTime startAt, LocalDateTime endAt, String location)
public void     ScheduleService.confirm(Long id, Long userId)

// 추가 - 읽기 전용, 기존 동작 영향 없음
public List<Notice>   NoticeService.findAll()                // noticeRepository.findAll() 위임, id 오름차순
public List<Schedule> ScheduleService.ownedBy(Long ownerId)   // scheduleRepository.findByOwnerId(ownerId) 위임

// 컨트롤러
@PostMapping                public NoticeResponse create(@RequestBody CreateNoticeRequest request)
@PostMapping("/{id}/publish") public void publish(@PathVariable Long id, @RequestBody PublishNoticeRequest request)
@GetMapping                 public List<NoticeResponse> list()
@PostMapping                public ScheduleResponse create(@RequestBody CreateScheduleRequest request)
@PostMapping("/{id}/confirm") public void confirm(@PathVariable Long id, @RequestBody ConfirmScheduleRequest request)
@GetMapping("/owner/{userId}") public List<ScheduleResponse> ownedBy(@PathVariable Long userId)

// DTO
public record CreateNoticeRequest(String title, String content, Integer category, Long writerId, Boolean pinned)
public record PublishNoticeRequest(Long userId)
public record NoticeResponse(Long id, String title, String content, int status, int category,
                             Long writerId, boolean pinned, LocalDateTime createdAt)
public record CreateScheduleRequest(String title, Long ownerId, LocalDateTime startAt, LocalDateTime endAt, String location)
public record ConfirmScheduleRequest(Long userId)
public record ScheduleResponse(Long id, String title, int status, Long ownerId,
                               LocalDateTime startAt, LocalDateTime endAt, String location)
```

설계 원칙: 컨트롤러는 DTO → 서비스 인자 변환과 엔티티 → 응답 DTO 변환만 한다. 게시 권한 · 상태 · 겹침 · 확정 규칙은 Notice · Schedule 엔티티와 서비스에 이미 있으므로(Notice.java:84-93, Schedule.java:61-117) 컨트롤러에 다시 쓰지 않는다. 서비스 생성자는 이미 생성자 주입(NoticeService.java:50-63, ScheduleService.java:34-38)이라 그대로 주입받는다.

5. 인터페이스 설계

기능	HTTP 메서드	URL(확정안)	요청 본문	성공 응답	서비스 호출
공지 작성	POST	/api/notices	CreateNoticeRequest	200 + NoticeResponse	NoticeService.create
공지 게시	POST	/api/notices/{id}/publish	PublishNoticeRequest	200 + 빈 본문	NoticeService.publish
공지 목록	GET	/api/notices	없음	200 + NoticeResponse 배열	NoticeService.findAll (추가)
일정 등록	POST	/api/schedules	CreateScheduleRequest	200 + ScheduleResponse / 거부 시 200 + 빈 본문	ScheduleService.create
일정 확정	POST	/api/schedules/{id}/confirm	ConfirmScheduleRequest	200 + 빈 본문	ScheduleService.confirm
내 일정	GET	/api/schedules/owner/{userId}	없음	200 + ScheduleResponse 배열(없으면 [])	ScheduleService.ownedBy (추가)

요청 필드

DTO	필드	타입	필수	생략 시
CreateNoticeRequest	title	string	아니오	null 그대로 저장(서비스 현행: 검증 없음)
	content	string	아니오	null 그대로
	category	int (1 일반 · 2 긴급 · 3 인사)	예	언박싱 NullPointerException → 500 (입력 검증은 REQ-NF-003)
	writerId	long	아니오	null 그대로(작성자 존재 확인 없음 - 레거시)
	pinned	boolean	아니오	false (compact 생성자)
PublishNoticeRequest	userId	long	예	null → findById(null) 예외 → 500
CreateScheduleRequest	title	string	아니오	null 이면 서비스가 null 반환 → 200 + 빈 본문
	ownerId	long	아니오	null 그대로 저장
	startAt · endAt	string(ISO-8601, 예 2026-10-20T10:00:00)	아니오	null 이면 서비스가 null 반환
	location	string	아니오	null 그대로

DTO	필드	타입	필수	생략 시
ConfirmScheduleRequest	userId	long	예	null → findById(null) 예외 → 500

응답 예시

```
POST /api/notices
{"title":"시스템 점검","content":"토요일 점검","category":2,"writerId":2,"pinned":true}
→ 200
{"id":1,"title":"시스템 점검","content":"토요일 점검","status":0,"category":2,
 "writerId":2,"pinned":true,"createdAt":"2026-10-17T09:00:00"}          // 필드 8개

POST /api/notices/1/publish   {"userId":2}   → 200, 본문 ""

POST /api/schedules
{"title":"주간회의","ownerId":1,"startAt":"2026-10-20T10:00:00","endAt":"2026-10-
20T11:00:00","location":"3층"}
→ 200
{"id":1,"title":"주간회의","status":0,"ownerId":1,
 "startAt":"2026-10-20T10:00:00","endAt":"2026-10-20T11:00:00","location":"3층"}   // 필드 7개

POST /api/schedules/1/confirm  {"userId":1}  → 200, 본문 ""
```

오류 동작

상황	응답(이 요구사항 범위)	근거
게시 · 확정 조건 불충족(권한 없음 · 상태 다름 · 없는 id · 없는 사용자)	200 + 빈 본문 (서비스가 조용히 무시)	NoticeService.java:78-95, ScheduleService.java:78-92 - 결재 process 와 같은 계약
일정 등록 거부(필수값 누락 · 시간 역전 · 겹침)	200 + 빈 본문 (서비스 null 반환을 그대로)	ScheduleService.java:46-51
필수 정수 필드 누락 · 경로 id 형식 오류	500 / 400 (스프링 기본)	입력 검증은 REQ-NF-003, 명시적 오류 응답은 REQ-NF-002 범위
소유자 null 일정 확정	NullPointerException → 500	Schedule.java:108-110 (레거시 결함 보존)

6. 데이터 설계

테이블 · 컬럼 · 저장값은 바꾸지 않는다. 응답 DTO 는 엔티티 필드를 그대로 옮기되 정수 code 를 유지한다.

엔티티(테이블)	컬럼	타입	코드값(DB 정수 그대로)	응답 DTO 필드
Notice (notice)	id · title · content · writer_id · pinned · created_at	Long · String · String · Long · boolean · LocalDateTime	-	같은 이름 (camelCase)
	status	int	0 임시 · 1 게시 · 9 내림 (Notice.java:23-25)	status (int)
	category	int	1 일반 · 2 긴급 · 3 인사 (Notice.java:35)	category (int)
Schedule (schedule)	id · title · owner_id · start_at · end_at · location	Long · String · Long · LocalDateTime x2 · String	-	같은 이름

엔티티(테이블)	컬럼	타입	코드값(DB 정수 그대로)	응답 DTO 필드
	status	int	0 예정 · 1 확정 · 9 취소 (Schedule.java:23-25)	status (int)

상태 전이표

대상	현재 상태	API	조건	다음 상태	부수효과
공지	0 임시	POST /{id}/publish	요청자 role ≥ 2 (User.java:40-42)	1 게시	저장 → (분류 2면 전체 사용자 메일) → 감사로그 NOTICE PUBLISH
공지	1 게시 · 9 내림	POST /{id}/publish	-	변화 없음	없음(조용히 무시)
일정	(없음)	POST /api/schedules	제목 · 시각 유효 + 같은 소유자 일정과 안 겹침	0 예정	저장 → 감사로그 SCHEDULE CREATE
일정	0 예정	POST /{id}/confirm	요청자 = 소유자	1 확정	저장 → 감사로그 SCHEDULE CONFIRM
일정	1 확정 · 9 취소	POST /{id}/confirm	-	변화 없음	없음

7. 처리 흐름

단계	공지 작성	공지 게시	일정 등록	일정 확정
1 요청 변환	CreateNoticeRequest → create 인자 5개	PublishNoticeRequest.userId	CreateScheduleRequest → create 인자 5개	ConfirmScheduleRequest.userId
2 가드	없음(레거시: 검증 없음)	공지 없음 · 사용자 없음 · notice.publish(user)=false → 종료	Schedule.isCreatable=false · 겹침 → null	일정 없음 · 사용자 없음 · schedule.confirm(userId)=false → 종료
3 상태 변경	Notice.draft (status 0)	status 0 → 1	Schedule.draft (status 0)	status 0 → 1
4 저장	noticeRepository.save	noticeRepository.save	scheduleRepository.save	scheduleRepository.save
5 메일	없음	isEmergency() 면 전체 사용자에게 [긴급공지] 메일	없음	없음
6 감사로그	NOTICE CREATE	NOTICE PUBLISH (메일 뒤)	SCHEDULE CREATE	SCHEDULE CONFIRM
7 응답	NoticeResponse.from	200 + 빈 본문	ScheduleResponse.from 또는 빈 본문(null)	200 + 빈 본문

근거 위치: NoticeService.java:66-71 · 78-95 · 98-103, ScheduleService.java:44-56 · 78-92. 2~6 단계는 기존 서비스가 수행하며 컨트롤러는 1 · 7 단계만 맡는다.

8. 예외 · 경계 처리

경계 · 예외	현재 서비스 동작	API 설계에서의 처리	레거시 보존 여부
사원(role 1)이 게시	조용히 무시, 로그 없음	200 + 빈 본문	보존
이미 게시 · 내림 공지 재게시	무시, 로그 1회만	200 + 빈 본문	보존
없는 공지 id · 없는 사용자	예외 없이 무시	200 + 빈 본문	보존
공지 id null	findById(null) 예외	경로 변수라 null 불가 - 해당 없음	-
긴급(2) 공지 게시	전 사용자 메일 후 감사 로그	그대로	보존
일정 제목 "" · null, 시각 null, 끝 < 시작	null 반환, 저장 · 로그 없음	200 + 빈 본문	보존(원인 구분은 REQ-NF-002)
시작 = 끝 · 공백 제목 " "	생성됨	그대로	보존
끝과 시작이 맞닿음	겹치지 않음 → 생성	그대로	보존
취소(9) 일정과 겹침	생성됨	그대로	보존
소유자 아닌 사용자 확정	무시	200 + 빈 본문	보존
소유자 null 일정 확정	NullPointerException	500	보존(레거시 결함 - 개선 대상)
requestBody 의 userId 를 그대로 신뢰	-	그대로 받음	보존(신원 확인은 REQ-SE-002)

9. 검증 설계

검증 기준	현재 테스트	추가 제안 테스트
API 로 등록한 공지 · 일정이 서비스 특성화 테스트와 같은 결과	테스트 없음	NoticeControllerTest#공지_작성_API는_임시_상태0으로_입력값_그대로_응답한다, ScheduleControllerTest#일정_등록_API는_예정_상태0으로_입력값_그대로_응답한다
응답 JSON 에 DTO 필드만	테스트 없음	NoticeControllerTest#공지_응답은_DTO_필드_8개만_나온다, ScheduleControllerTest#일정_응답은_DTO_필드_7개만_나온다
(설계 추가) 무시 동작 보존	서비스 수준: NoticeServiceCharacterizationTest#사원이_게시하면_조용히_무시되고_상태0_그대로다, ScheduleServiceCharacterizationTest#소유자가_아닌_사용자가_확정하면_조용히_무시된다	NoticeControllerTest#사원이_게시_API를_호출하면_200과_빈_본문이고_상태는_0이다, ScheduleControllerTest#겹치는_일정_등록은_200과_빈_본문이다

```
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class NoticeControllerTest {
    private static final String CREATE_REQUEST = """
            {"title":"시스템 점검","content":"토요일 점검","category":2,"writerId":2,"pinned":true}""";

    @Autowired
    private MockMvc mockMvc;
```

```
    @Test
    void 공지_응답은_DTO_필드_8개만_나온다() throws Exception {

mockMvc.perform(post("/api/notices").contentType(MediaType.APPLICATION_JSON).content(CREATE_REQUEST))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.*", hasSize(8)))
                .andExpect(jsonPath("$.status").value(0))
                .andExpect(jsonPath("$.category").value(2));
    }

    @Test
    void 사원이_게시_API를_호출하면_200과_빈_본문이고_상태는_0이다() throws Exception {
        String responseJson = mockMvc.perform(post("/api/notices")

.contentType(MediaType.APPLICATION_JSON).content(CREATE_REQUEST)).andReturn().getResponse().getContentAsS
tring();
        long noticeId = ((Number) JsonPath.read(responseJson, "$.id")).longValue();

        mockMvc.perform(post("/api/notices/" + noticeId + "/publish")
                        .contentType(MediaType.APPLICATION_JSON).content("{\"userId\":1}"))
                .andExpect(status().isOk())
                .andExpect(content().string(""));
        mockMvc.perform(get("/api/notices"))
                .andExpect(jsonPath("$[?(@.id == " + noticeId + ")].status").value(0));
    }
}
```

기존 NoticeServiceCharacterizationTest(17개) · ScheduleServiceCharacterizationTest(24개)는 본문을 수정하지 않고 그대로 green 이어야 한다.

10. 검토 의견 · 후속 과제

보존 대상

- 기존 NoticeService · ScheduleService public 시그니처와 업무 규칙(게시 권한 팀장 이상, 겹침 경계 '미만', 확정은 소유자만)
- 감사로그 문구 NOTICE CREATE · NOTICE PUBLISH · SCHEDULE CREATE · SCHEDULE CONFIRM 과 형식 [AUDIT] [yyyy-MM-dd HH:mm:ss] 동작 id=.. by=..
- 긴급공지 메일 제목 [긴급공지] {제목} · 본문 양식(MailFormat), 메일 후 감사로그 순서
- 조용한 무시 · null 반환, 소유자 null 확정 시 NullPointerException
- DB 저장값 status 0/1/9 · category 1/2/3 정수

개선 대상 (차후 재설계)

- 조용한 무시 · null 반환 → 사유 있는 오류 응답(404/403/409/400) - REQ-NF-002
- 필수 필드 누락 시 500 → 400 + 필드별 사유 - REQ-NF-003
- 본문 userId 신뢰 → 로그인 사용자 - REQ-SE-002
- 공지 status · category, 일정 status 를 enum 으로 (DB 정수 유지) - REQ-DA-003
- 범위 밖: UserController 가 User 엔티티를 직접 반환(UserController.java:23-25) - 같은 DTO 원칙 적용 검토

검토 의견

F열의 목록 · 내 일정 조회는 '기존 서비스 그대로 호출' 조건과 달리 서비스에 해당 메서드가 없다. 업무 규칙이 없는 단순 조회이므로 서비스에 읽기 메서드(findAll · ownedBy)를 추가하는 것으로 해석하며, 이는 기존 동작을 바꾸지 않는다. 일정 등록 거부 시 응답은 결재 process 와 맞춰 200 + 빈 본문으로 두고, 오류 응답 전환은 REQ-NF-002 에서 한 번에 바꾼다.

연계 요구사항: REQ-NF-002(명시적 오류), REQ-NF-003(입력 검증), REQ-SE-002(요청자 신원), REQ-DA-003(코드 enum), REQ-NF-001(결재 API 계약 - 같은 DTO 방식).